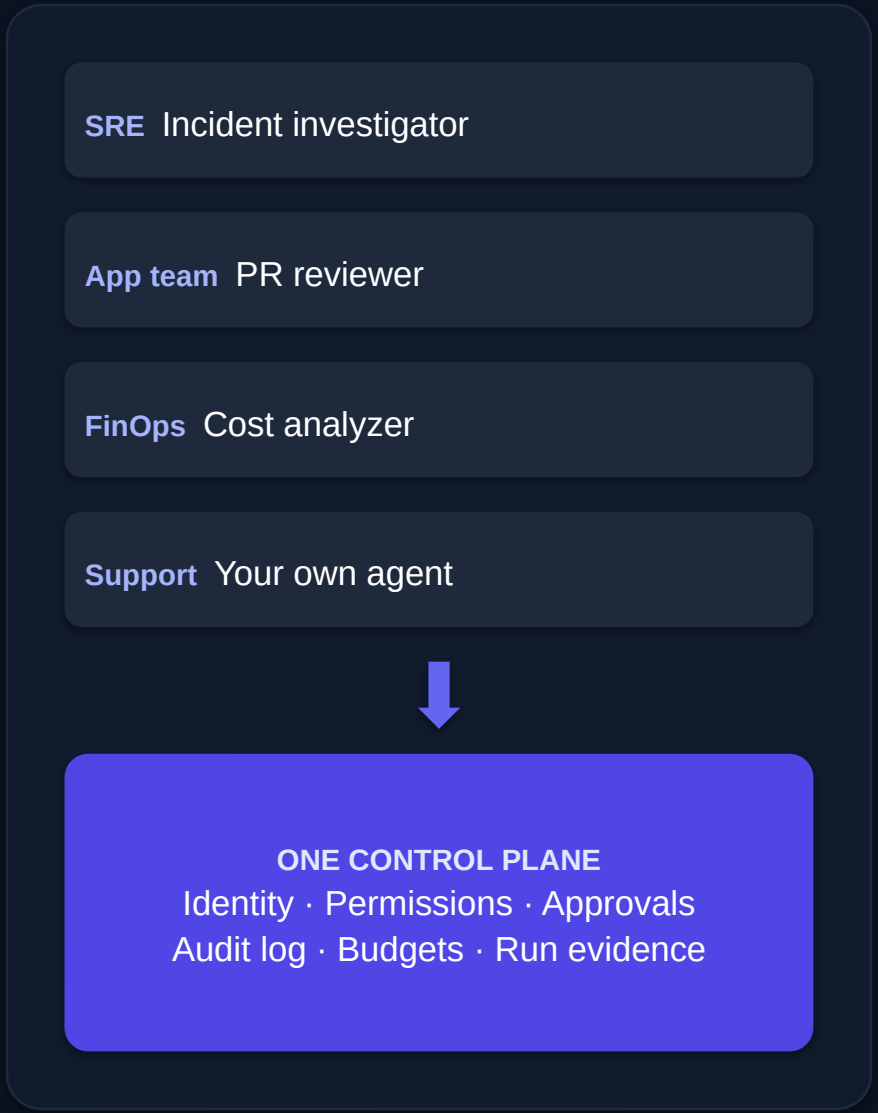
KOMODOR AGENTIC OPERATIONS PLATFORM

Your teams are already shipping agents.

Can you govern them?

KAOP gives every agent that touches production an identity, a boundary, an approval path and a record — so the platform team can say yes to more of them.



A conversation for platform engineering leaders

Agents are spreading faster than your operating model

Every team's agent is a reasonable decision. The risk sits between them.

NO INVENTORY

Who owns it?

Agents arrive team by team. Nobody can list what is running, who owns it, or what it costs.

INCONSISTENT ACCESS

What can it touch?

Each agent brings its own credentials. Limits live in prompts, where nothing enforces them.

NO SINGLE RECORD

What did it do?

The alert, the investigation and the code change live in three tools. Reconstructing a decision is manual work.

Gartner expects over 40% of agentic AI projects to be cancelled by 2027 for governance gaps, unclear value or rising cost.

Cited in Komodor's KAOP launch, September 2026

ASK Which of these three is costing your platform team the most today?

KAOP runs agents like production services

One platform to build, run, govern and optimize every agent, Komodor's and your own.

BUILD

Start ready, or bring your own

Komodor-built catalog agents: Kubernetes RCA, Grafana and Datadog investigators, Code Reviewer, remediation. Or import agents built on LangChain, the Claude Agent SDK or ADK.

RUN

Wire agents to real triggers

Alert endpoints, schedules and Slack start runs. Workflows fan work out to specialists. Agents run in Komodor cloud or inside your network, dialing out only.

GOVERN

Enforce, don't instruct

Every agent has its own identity and default-deny permissions. Guardrails block or redact. Risky actions wait for a person. Everything lands in an append-only audit log.

OPTIMIZE

Keep what earns its cost

Every run records inputs, tool calls, reasoning, output and cost. LLM-judge evaluations grade quality, and budgets cap spend per agent.

FOUNDATION Klaudia's Kubernetes root-cause expertise, packaged as agents your teams deploy in minutes.

A prompt is not a permission model

KAOP governs an agent the same way it governs a person, by the same machinery.

	LAYER	QUESTION IT ANSWERS	WHAT IT STOPS
1	**Identity**	Who is acting?	An action nobody is accountable for
2	**Permissions**	Allowed on this resource?	An identity reaching beyond its job
3	**Standards**	Built to our rules?	An agent with no owner, budget or grading
4	**Guardrails**	May this action run now?	A permitted tool used destructively
5	**Approvals**	Should a person decide?	An irreversible action with no human
6	**Audit log**	What happened, and who?	A change nobody can reconstruct

IN THIS LAB

Kubernetes RCA
Reads the cluster through Kubernetes' own view role. No Secrets, no exec, no writes.

GitHub Agent
Opens one PR for one config value, and only after I approve it. It never merges.

Code Reviewer
Posts one comment. It cannot approve, so branch protection still needs a human.

PRINCIPLE If I claim an agent is read-only, I show the RBAC that makes it so, not the prompt that asks it to be.

Visibility: one record from the alert to the merged fix

What the platform team sees without asking any other team.

Fleet health

Every agent: online, quiet or failing, who owns it, and what changed.

Runs & evidence

Inputs, tool calls, reasoning, output and cost for every run, workflow steps included.

Attention inbox

Approvals waiting on a person, with the evidence needed to decide.

Audit log

Append-only record of governed changes and sensitive reads, with actor and outcome.

Spend

Cost per run, agent, model and team, and budgets that block new runs.

Access explorer

"What can this agent actually do?" answered before you grant anything.

FOR YOU Inventory, access and audit stop being a quarterly spreadsheet and become a screen.

Today: one incident, two governed workflows

Built from KAOP catalog agents. No custom RCA engine, no prompt engineering.

WORKFLOW 1 · INCIDENT · owned by SRE · starts on a Grafana alert

Grafana alert
API restarting

→

Investigation
K8s RCA + Grafana Investigator

→

Remediation
Decides: fix, no action or questions

→

Human approval
Attention inbox

→

GitHub Agent
Opens one PR

WORKFLOW 2 · CODE REVIEW · owned by the app team · starts on any pull request

Pull request
From the agent or a person

→

Code Reviewer
One comment per commit

→

Human merge
Branch protection

→

Deploy from git
Argo/Flux in production

→

Verify
Same outage, zero restarts

SHARED One fleet, one policy model, one evidence trail. Two triggers, two owners, two separately scoped identities.

Who can do what, and what enforces it

Least privilege per agent, enforced outside the agent.

Agent	Can read	Can change	Enforced by
Kubernetes RCA in the lab cluster	Kubernetes view role: pods, events, logs, workloads	Nothing: no Secrets, no exec, no writes	Kubernetes RBAC on its own ServiceAccount
Grafana Investigator Komodor cloud	Metrics, events, alert state	Nothing in Grafana	Dedicated Grafana Viewer token
Remediation Komodor cloud	The finished investigation	Nothing	Holds no credentials for any system
GitHub Agent Komodor cloud	Repository code and history	One PR, one config value, after approval	KAOP approval gate + GitHub App scopes
Code Reviewer Komodor cloud	The PR diff	One comment per commit	GitHub App scopes; cannot approve or merge

VERIFIED KAOP's agent chart RBAC, applied to this cluster and checked with kubectl auth can-i: reads pods, events, logs; denied Secrets, exec, delete, patch, scale.

Live: one bad rollout, one Redis outage

Real cluster, real alert, real agents. Nothing on screen is staged.

1 Latent defect

Last week's rollout points liveness at /readyz, which depends on Redis.

2 Dependency blip

Redis goes away. Readiness fails, which is correct.

3 Amplified

Liveness fails too, so Kubernetes kills a healthy API again and again, and pages the wrong team.

4 Detect

Grafana's restart alert fires and posts to KAOP.

5 Investigate

Root cause, confidence and evidence from two specialists.

6 Decide

I approve. The GitHub Agent opens the one-line fix.

7 Review

The Code Reviewer comments. A human merges. The re-run shows zero restarts.

MEASURED IN REHEARSAL

Bad probe
5 kills in 5 min; alert at +3 min

Fixed probe
0 kills; alert stayed Normal

WATCH FOR The trigger (Redis went away) versus the root cause (a probe that punishes the API for it).

Start with one owned workflow, and measure it

A two-week pilot your platform team runs, with pass criteria agreed up front.

WEEK 1

Incident investigation

One cluster, read-only Kubernetes RCA, alerts from your existing Grafana or Datadog. Approvals on every write.

WEEK 2

Add PR review

One repository on the Code Reviewer. Agent-opened fixes go through your normal review and merge.

DECISION

Scale or stop

Review the scorecard together. Expand to more teams only if the numbers hold.

PILOT SCORECARD · proposed thresholds, agreed with you

Evidence

Each RCA cites the change and two or more evidence types

Usefulness

On-call rates the RCA actionable in 4 of 5 incidents

Control

Zero out-of-policy tool calls; every write approved

Cost

Cost per run within the agreed budget

ASK Which workflow would you sponsor first, and who on your team owns it?

How I built this pitch

Every claim traces to Komodor's own material or to something run in this lab.

Komodor KAOP launch (Sept 2026)	Positioning: build, run, govern, optimize. Gartner's cancellation stat and the CTO's line that building the agents isn't the hard part.
KAOP docs: Security & governance	The six governance layers, approvals, audit log and Access explorer. Slides 4 and 5.
KAOP docs: Agent catalog	Kubernetes RCA, Grafana Investigator, Remediation, GitHub Agent and Code Reviewer: what each reads, changes and where it stops. Slides 6 and 7.
KAOP docs: Incident management module	The Production Alert Investigation workflow: source, correlate, investigate, remediate.
Klaudia announcement (2024)	Why the RCA is credible: Komodor's existing root-cause expertise, now packaged as an agent.
This lab	A real fault, real Grafana alert, RBAC verified with kubectl, recovery measured with and without the fix.

STRATEGY Lead with the platform team's pain, prove it with one real incident, close on a measurable pilot.

Questions I expect, and my answers

"We'll build this ourselves."	You can build an agent. Building the identity, approvals, audit, budgets and evaluation around every agent, for every team, is the platform. That's KAOP.
"Komodor already has Klaudia. Why KAOP?"	Klaudia is the expertise. KAOP is the operating model: it runs Klaudia-grade agents beside your own, under one policy and one record.
"What if the agent is wrong?"	Remediation separates observed evidence from inference and can answer no action. Every write needs approval. Evaluations grade runs over time.
"Will it touch production?"	Only through actions you approve. In this lab, no agent can write to the cluster, and the fix went through git and a human.
"What does it cost?"	Every run shows its cost. Budgets block new runs at a cap. The pilot measures cost per useful RCA.

Rehearsal evidence

Same Redis outage, measured with both probe configurations.

	Liveness on /readyz (bad rollout)	Liveness on /healthz (fix)
Liveness kills of a healthy API	5 in a 5-minute outage	0 in a 2.5-minute outage
Liveness HTTP 503 events	Yes	None
Grafana restart alert	Pending +111 s, Alerting +175 s	Stayed Normal
API Ready after Redis returned	4.8 s (between back-offs)	6.5 s
Previous container log	ConnectionRefusedError in /readyz	n/a, no restart

Rehearsal on 2026-10-24, single-node kind cluster. With the bad probe, recovery depends on where the restart back-off is when Redis returns (10 s doubling to 5 min). Raw evidence: artifacts/incidents/; summary: docs/evidence.md.